

Four deliverables, sequenced below as **D1** → **D4**. Nearly all of the work is **extension of shipped code**, not greenfield — the factory, six flow templates, the builder canvas and a payroll machine API already run on testnet. Net-new concentrates in three places: the **Soroswap router call**, the **versioned `/api/v1` surface**, and the **shared input primitives**. Full detail in docs/instawards-phase-1.md.

D1 · Real DEX swap

SOROBAN / RUST — EXTEND + NET NEW · WEEK 1

Wire the existing swapper to the Soroswap testnet, unhide it in the builder, and produce a verifiable swap transaction. Today it fulfils swaps from a pre-funded balance — its own comment says “*here we simulate.*”

WHAT GETS BUILT

- Contract-to-contract authorization** — `authorize_as_current_contract` so the router can pull `asset_in`. No crate here does this yet, and it is the one piece unit tests cannot prove.
- The router call** — quote via `router_get_amounts_out`, derive `amount_out_min` from `slippage_bps`, deadline-bounded, revert on shortfall.
- parent + require_auth** — swapper and yield are the only two pipeline contracts missing it, so `receive_and_forward` is callable by anyone today.
- Constructor arity fix** — `scval.ts` sends 6 args where Rust takes 5. Swap flows cannot deploy at all today; the same bug sits in `yield`.
- Schema move** — `rateBps` → `slippageBps` + `routerAddress`, with a `migrateFlowGraph` branch so saved flows still load. Router is an allowlist, never free text.

DONE WHEN a flow built on the canvas deploys to testnet and a triggered swap routes through the *real* Soroswap router — verifiable on `stellar.expert`, not a mock.

D2 · Partner-drivable API

TYPESCRIPT / NEXT — EXTEND + NET NEW · WEEK 2

Today Paiflow is a thing people use. D2 makes it a thing *software* uses. Payroll already has a machine interface, but it is one-off and payroll-specific.

THE V1 SURFACE

Execute	POST <code>/api/v1/deployments/[id]/execute</code> — relayer-signed, no user signature at request time
Events	GET <code>.../events</code> — cursor-paged, reusing the payroll (<code>occurredAt</code> , <code>id</code>) cursor
Tokens	Hashed, per-deployment. Closes open security issue #247
Wrapper	<code>withApiV1</code> — auth + rate limit + audit, extracted from payroll
Docs	OpenAPI + Postman, hand-written — no spec tooling exists in the repo

Load-bearing decision: the swapper has no `execute` endpoint and the relayer cannot call it directly. The one path that works fronts the flow with a webhook contract, whose `execute_escrow` needs *only* relayer auth: the user pre-funds once, the partner then fires swaps with no signature. Zero contract changes — but the endpoint executes a *pipeline*, not a swapper.

DONE WHEN an authenticated `curl` from outside the app causes a real swap on testnet and returns a tx hash, and the events endpoint pages through that deployment's events.

D3 · Reusable builder inputs

REACT / TSX — EXTENDS SHIPPED CODE · WEEK 3

The builder ships today. This deliverable is consolidation, not construction: 14 node types and 14 hand-written config forms live in one 2,800-line file, each inventing its own way to ask for an amount, an address, or a percentage.

WHAT GETS BUILT

`AmountInput` and `ShareInput` are new — hand-rolled at 8 sites and copy-pasted verbatim twice. `AssetSelect` is an extraction of an existing file-private component; `AddressPicker` already ships and is used 7×. A `jsdom` + RTL component harness has to exist before any of it can be tested.

DESIGN POSTURE

- Test harness first.** A `.test.tsx` is not collected by the current runner — it reports green having run nothing. Prove CI goes red before trusting it.
- Build four, adopt what fits.** The Swapper panel has no amount field, so `AmountInput` is proven only if adoption widens to the split panel. Name that choice.
- Lossless bps.** The split's share input rounds 5 bps to 10 — reused as-is on slippage it silently doubles the user's tolerance.

DONE WHEN the Swapper panel renders shared primitives, error markup is consistent, and component tests run green in CI *on a runner that collects them*.

D4 · Validation package

VIDEO · GUIDE · EVIDENCE — NET NEW · WEEK 4

The Chapter Lead does not review pull requests. They review evidence, against the grid in SOW §6.2 — which carries a row for this package alongside D1–D3. Work finished in the repo but unevidenced is, for this award, not finished.

ARTIFACTS

- 3–5 min technical demo video** — published, cross-linked from the repo.
- Integration guide** — `curl` examples and sample XDR payloads. Written *during* D2: if they are painful to write, the API is painful to use.
- Receipt list** — every tx hash on Stellar Expert, plus contract and WASM hashes.
- Screen recordings** — deploy, trigger, and the API-driven swap.

The numbers it carries: ≥5 unique flows · ≥6 distinct wallets · ≥60 on-chain events · ≥5 swapper executions · ≥1 WASM uploaded · live testnet URL. Most of the counting is already automated by `/admin/submission-proof` — three added counts finish it.

DONE WHEN the Chapter Lead can verify all four rows of §6.2 without asking a follow-up question.

TIMELINE — WHERE THE SLACK IS

160 hours across 30 days. 123 h development + 37 h QA is ~6 focused hours every working day for a month — one person, no bus factor, no slack for a bad week.

Week 4 holds 9 calendar days against a ~4-day task list. That is the entire buffer. The cut is named before the sprint starts: hand-write the OpenAPI spec rather than adding generation tooling, then D3's mobile pass.

CRITICAL PATH — ONE GATE DOMINATES

D1 palette unhide → D3 build a swap flow → D3 evidence
D1 router auth → D1 swap tx → D2 execute demo → D4 video

The swap block is hidden, so **no swapper flow can be built at all** — which silently gates every D3 artifact. It is a one-line change plus 8 AI-prompt edits, and it lands Day 1, before the DEX call works.

Router authorization gates everything else. Both are answerable on Day 1.

TOP RISKS

- Contract-to-contract auth → day-1 spike; only a testnet tx counts
- No Soroswap pool for our pair → confirm Day 1, else seed one
- Evidence captured late → capture as it happens, assemble in W4
- `.test.tsx` silently skipped → prove CI goes red first
- 6 wallets / 60 events don't accrue → recruit testers in W1
- Unnamed work (~12 items) → the build sheet is the real checklist